

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation
Weightage: 20%

Code of Conduct:

I T. Nikhil Kumar Reddy (1923720234) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Nikhil Kumar Reddy

QUESTIONS:

Total Marks: 20

1. Servlet Architecture & Workflow

Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

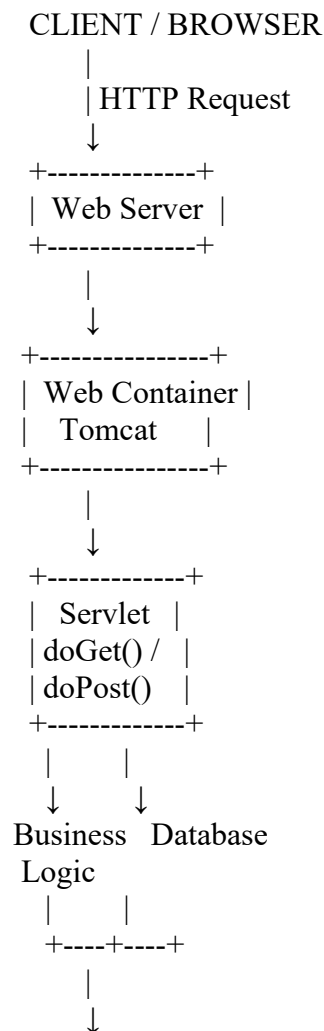
Focus Areas:

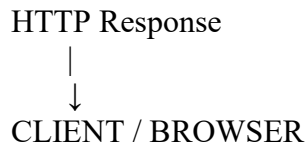
- Client–Server interaction
- Role of Web Container
- Request and Response objects

Introduction

A **Java Servlet** is a server-side Java program used to handle client requests and generate dynamic responses. Servlets run inside a **Web Container** such as Apache Tomcat. The container manages the servlet life cycle, request processing, response generation, and session management.

Servlet Architecture



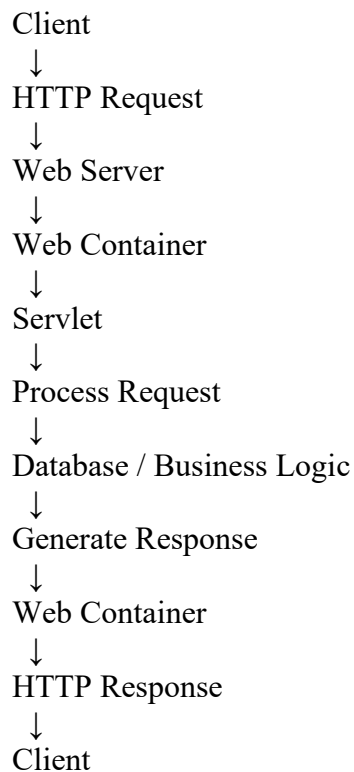


Role of Web Container

The Web Container is responsible for:

1. Loading and creating servlet objects.
2. Managing the servlet life cycle.
3. Creating `HttpServletRequest` and `HttpServletResponse` objects.
4. Calling `service()`, `doGet()` and `doPost()`.
5. Managing sessions and security.
6. Sending the generated response back to the client.

Request–Response Cycle



Request Object

`HttpServletRequest` contains information sent by the client.

Examples:

- Form parameters
- HTTP headers
- Cookies
- Session information
- Request method

Example:

```
String name = request.getParameter("name");
```

Response Object

HttpServletResponse is used by the servlet to send information back to the client.

Example:

```
response.setContentType("text/html");PrintWriter out =  
response.getWriter();out.println("Welcome " + name);
```

Conclusion

Servlet architecture provides an efficient mechanism for processing client requests and generating dynamic server-side responses.

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of init(), service(), and destroy() methods.

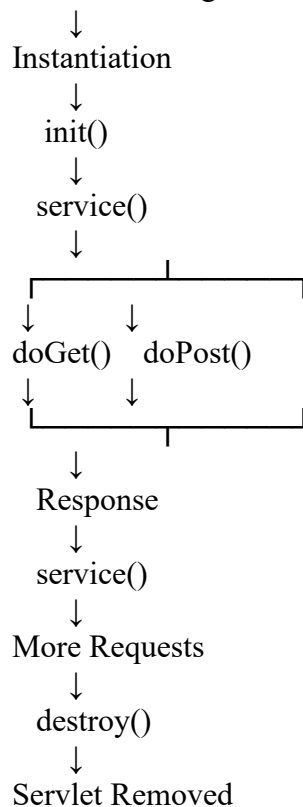
Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

The **Servlet Life Cycle** describes the different stages through which a servlet passes from creation to destruction. The Web Container controls the complete life cycle.

Servlet Life Cycle

Servlet Loading



1. init()

The init() method is called **only once** when the servlet is initialized.

It is used for:

- Initializing resources
- Reading configuration
- Establishing required connections
- Performing one-time setup

Example:

```
public void init() {
    System.out.println("Servlet Initialized");
}
```

2. service()

The service() method is called for **every client request**.

It determines the HTTP request method and calls the appropriate method.

GET → doGet()

POST → doPost()

Example:

```
protected void service(HttpServletRequest request,
                        HttpServletResponse response) {
    // Process request
}
```

3. destroy()

The destroy() method is called once before the servlet is removed from memory.

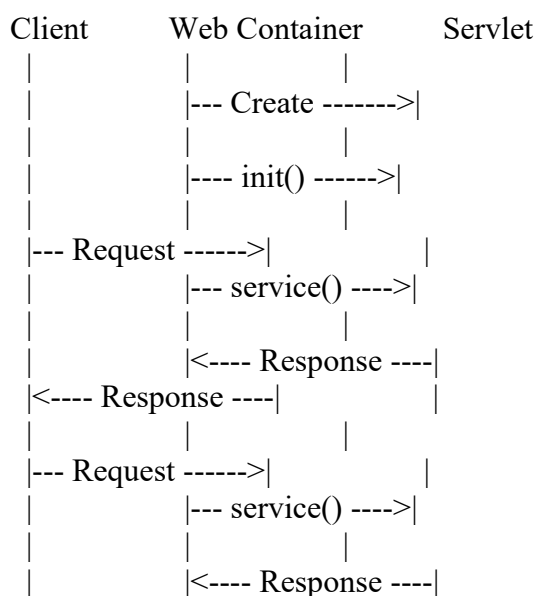
It is used to:

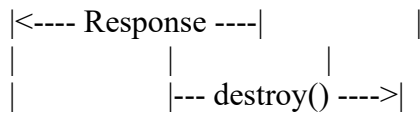
- Close database connections
- Release resources
- Stop background operations
- Perform cleanup

Example:

```
public void destroy() {
    System.out.println("Servlet Destroyed");
}
```

Sequence Diagram





Performance Implications

- `init()` should contain only one-time operations.
- `service()` must be efficient because it is executed for every request.
- Heavy processing inside `service()` can reduce performance.
- Resources should be properly released in `destroy()`.

Conclusion

Proper understanding of the servlet life cycle helps developers create efficient, scalable, and reliable web applications.

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations

GET and POST are HTTP methods commonly used for submitting information from a client to a server.

Feature	GET	POST
Data transmission	URL	Request body
Visibility	Visible in URL	Not visible in URL
Data size	Limited by URL length	Supports larger data
Bookmark	Can be bookmarked	Normally not bookmarked
Caching	Can be cached	Usually not cached
Main purpose	Retrieve data	Submit/change data
Sensitive data	Not suitable	More suitable
Server state	Usually does not change state	Can change server state

GET Example

`http://example.com/search?name=java`

In GET, parameters are appended to the URL.

Servlet:

```

protected void doGet(HttpServletRequest request,
    HttpServletResponse response) {

    String name = request.getParameter("name");
}
  
```

Suitable Applications for GET

- Searching products

- Viewing a webpage
- Filtering information
- Retrieving records
- Fetching public information

POST Example

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response) {

    String username = request.getParameter("username");
    String password = request.getParameter("password");
}
```

Suitable Applications for POST

- Login
- Registration
- Payment submission
- Uploading files
- Creating database records
- Updating information

Security Comparison

GET is not appropriate for sensitive data because the data can appear in:

- Browser history
- Server logs
- URLs
- Bookmarks

POST places the data in the request body, but **POST alone does not provide encryption.**

For secure communication:

HTTPS + POST + Server-side Validation

should be used when submitting sensitive information.

Conclusion

GET should mainly be used for **retrieving information**, while POST should be used when **submitting or modifying information**.

4. Session Tracking Techniques

Question:

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting).
Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

HTTP is a **stateless protocol**. This means the server does not automatically remember previous requests from the same user.

Session tracking techniques allow web applications to maintain user information across multiple requests.

Major Session Tracking Techniques

1. HttpSession
 2. Cookies
 3. URL Rewriting
 4. Hidden Form Fields
-

A. HttpSession

HttpSession stores user-related session information on the server.

Example:

```
HttpSession session = request.getSession();
session.setAttribute("username", "Mokshagna");
String name =
    (String) session.getAttribute("username");
```

Advantages

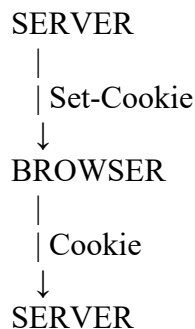
- Data is stored on the server.
- Easy to use with Servlets.
- Suitable for logged-in users.
- Can store multiple attributes.

Limitations

- Requires server resources.
 - Sessions need timeout management.
 - Session hijacking is possible if security is weak.
-

B. Cookies

A cookie is a small piece of information stored in the user's browser.



Example:

```
Cookie cookie =
    new Cookie("username", "Mokshagna");
response.addCookie(cookie);
```

Advantages

- Easy to implement.
- Reduces server-side storage.
- Useful for user preferences.
- Can persist across requests.

Limitations

- Users can disable cookies.
- Cookies can be stolen if not properly protected.
- Storage capacity is limited.

Security attributes include:

HttpOnly

Secure

SameSite

C. URL Rewriting

In URL rewriting, session information is added to the URL.

Example:

`https://example.com/home;jsessionid=ABC123`

Advantages

- Can work when cookies are disabled.
- Simple alternative session mechanism.

Limitations

- Session ID becomes visible in the URL.
 - URLs may be stored in browser history and logs.
 - Less secure than properly protected cookies.
 - Makes URLs longer.
-

Comparison

Technique	Storage	Advantage	Limitation
HttpSession	Server	Convenient and suitable for sessions	Uses server resources
Cookies	Client	Simple and efficient	Can be disabled/stolen
URL Rewriting	URL	Works without cookies	Session ID exposed
Hidden Fields	HTML page	Simple for forms	Limited to form navigation

Conclusion

HttpSession is commonly preferred for maintaining server-side user state, while cookies are often used to maintain the session identifier.

5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
- Security risks
- Performance considerations

Cookies and sessions are both used to maintain state in web applications, but their storage and security mechanisms are different.

Cookies

Cookies are stored on the **client side**, normally in the web browser.

Examples:

- Language preference
- Theme preference
- User preferences
- Session identifier

Sessions

Session data is maintained on the **server side**.

Examples:

- Logged-in user information
- Shopping cart data
- Temporary user information
- Authentication state

Cookies vs Sessions

Feature	Cookies	Sessions
Storage	Client/browser	Server
Security	More exposed to client	Server-side data is less directly exposed
Size	Small	Depends on server resources
Server memory	Low	Higher
Lifetime	Can be persistent	Usually timeout-based
Performance	Fast client access	Requires server-side lookup
User control	User can delete/disable	Controlled mainly by server
Best use	Preferences, identifiers	Login and temporary user data

Real-Time Use Case 1 – E-Commerce

Consider an online shopping website.

User Login



Create Session



Store User Information



Browse Products



Add Products to Cart



Checkout



Logout



Destroy Session

The session can maintain:

User ID

Cart ID

Login Status

A cookie can store a session identifier or user preferences.

Real-Time Use Case 2 – Online Banking

After successful login:

Login



Authentication



Create Secure Session



Access Account



Perform Transactions



Logout



Invalidate Session

Sensitive account information should not simply be stored in client-side cookies.

Security Risks

Cookie Risks

- Cookie theft
- XSS-related exposure
- CSRF
- Session hijacking

Session Risks

- Session hijacking
- Session fixation
- Weak session identifiers
- Improper session timeout

Security Measures

1. Use HTTPS.
 - Use HttpOnly cookie
2. s.
3. Use Secure cookies.
4. Use appropriate SameSite settings.

5. Regenerate session identifiers after authentication when appropriate.
 6. Invalidate sessions during logout.
 7. Use proper session timeout.
- Validate all user input on the server.
-

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals

Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism ; minimal contribution or ethical awareness